

MAST32001 Computational Statistics

Zhi Zhou

2026-09-27

Exercise 1 — Is there serial correlation?

The annual Nile river-flow series is available in R as Nile. We want to investigate whether consecutive observations show positive serial dependence.

```
knitr::opts_chunk$set(  
  echo = TRUE,  
  message = FALSE,  
  warning = FALSE,  
  fig.width = 10,  
  fig.height = 3.8  
)  
library(boot)  
library(ggplot2)  
library(ggpubr)  
data(Nile)  
set.seed(092501)
```

Task 1

Plot the series and its sample autocorrelation function.

Answer:

```
df <- data.frame(year = as.numeric(time(Nile)), flow = as.numeric(Nile))  
mean_flow <- mean(df$flow)
```

```

p_ts <- ggplot(df, aes(year, flow)) +
  geom_line(color = "steelblue", linewidth = 0.6) +
  geom_point(color = "steelblue", size = 1.3) +
  geom_hline(
    yintercept = mean_flow,
    color = "tomato",
    linetype = "dashed",
    linewidth = 0.7
  ) +
  annotate(
    "text",
    x = 1969,
    y = mean_flow - 400,
    label = sprintf("mean = %.1f", mean_flow),
    hjust = 1.1,
    vjust = -0.6,
    color = "tomato",
    fontface = "bold",
    size = 3.5
  ) +
  labs(title = "Annual Nile river flow (1871-1970)",
        x = "Year",
        y = expression(Flow ~ (10^8 ~ m^3))) +
  theme_pubr()

a <- acf(Nile, plot = FALSE)
band <- 1.96 / sqrt(length(Nile))
adf <- data.frame(lag = as.numeric(a$lag), acf = as.numeric(a$acf))
adf$sig <- adf$lag > 0 & abs(adf$acf) > band

p_acf <- ggplot(adf, aes(lag, acf, fill = sig)) +
  geom_col(width = 0.55, show.legend = FALSE) +
  scale_fill_manual(values = c("TRUE" = "tomato", "FALSE" = "steelblue")) +
  geom_hline(
    yintercept = c(band, -band),
    linetype = "dashed",
    color = "grey60"
  )

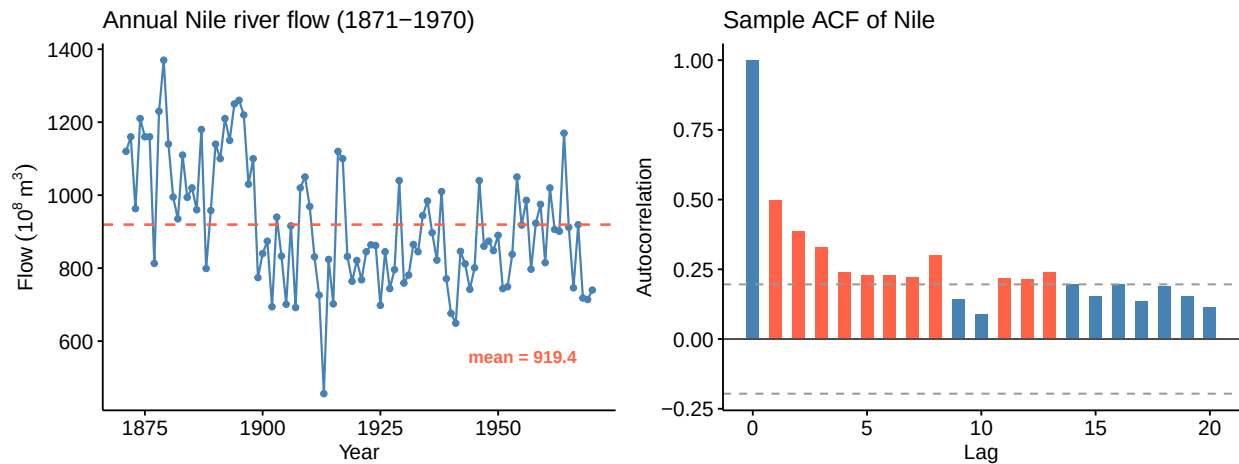
```

```

) +
geom_hline(yintercept = 0, color = "grey30") +
labs(title = "Sample ACF of Nile", x = "Lag", y = "Autocorrelation") +
theme_pubr()

ggarrange(p_ts, p_acf, nrow = 1, widths = c(1, 1))

```



The series moves around the mean, with a visibly drop after about 1900. The ACF starts at 0.50 and stays above the 95% iid band for several lags, dying out around lag 10.

Task 2

Use the lag-one sample correlation

$$T = \text{cor}(X_1, \dots, X_{n-1}, X_2, \dots, X_n)$$

as a test statistic for

$$H_0 : \text{the observations are iid}$$

against positive serial dependence.

Answer:

```

n <- length(Nile)
Tobs <- cor(Nile[-n], Nile[-1])
Tobs

```

[1] 0.5050531

The lag-one sample correlation is $T_{obs} = \text{cor}(X_1, \dots, X_{99}, X_2, \dots, X_{100}) = 0.5050531$. Under null hypothesis, T is approximately $N(0, 1/n) = N(0, 0.01)$.

Task 3

Consider the following possible ways to obtain a null distribution for T :

- a. randomly permute the observed values;
- b. ordinary iid bootstrap for the observed values;
- c. moving-block bootstrap;
- d. fit an AR(1) model to the original series and simulate from the fitter model.

For each model, decide whether it is appropriate for this particular null hypothesis. Explain your answer before doing any simulation.

Answer:

Only (a) is the right null. Permuting the series shuffles the order while keeping the same values, and under iid every ordering is equally likely. The iid bootstrap (b) is also valid, but it resamples with replacement instead of permutes, so it is a little less sharp. The moving-block bootstrap (c) is wrong because it is built to keep dependence, not to destroy it. Fitting an AR(1) and simulating (d) is also wrong: the fitted phi is about 0.50, so the simulated series is dependent too. That is an AR(1) null, not iid.

Task 4

Carry out a permutation test with $B = 4999$ random permutations. Use

$$p = \frac{1 + \sum_{b=1}^B I(T_b^* \geq T_{obs})}{B + 1}.$$

Answer:

```

B <- 4999
Ts <- replicate(B, {
  xs <- sample(Nile)
  cor(xs[-n], xs[-1])
})
k <- sum(Ts >= Tob)
p <- (1 + k) / (B + 1)
c(threshold_count = k, p_value = p)

```

threshold_count	p_value
0e+00	2e-04

No permuted values reached $T_{obs} = 0.5051$, so

$$p = (1 + 0)/(4999 + 1) = 0.0002.$$

We reject H_0 at the 5% and 1% levels.

Task 5

Interpret the result. What exactly does rejection tell us, and what does it not tell us?

Answer:

Rejection means the order matters: nearby years are not independent. It does not say what kind of dependence it is. An AR(1), long memory, a slow trend, or the 1899 dam change all look similar on the ACF. The 0.5051 is just this sample's estimate, not the true correlation. It also says nothing about the marginal distribution.

Task 6

Suppose instead that the goal were to estimate the standard error of the sample mean while allowing serial dependence. Would the same permutation scheme be appropriate? Which resampling idea from the lectures would be more natural?

Answer:

```

naive <- sd(Nile) / sqrt(n)

# Newey-West long-run variance (Bartlett, m = 20)
xc <- Nile - mean(Nile)
m <- 20
gammas <- sapply(1:m, function(k)
  sum(xc[1:(n - k)] * xc[(k + 1):n]) / n)
w <- 1 - (1:m) / (m + 1)
nw_se <- sqrt((sum(xc^2) / n + 2 * sum(w * gammas)) / n)

# Moving-block bootstrap
mb_mean <- function(d, idx)
  mean(d[idx, 1])
boot_mb <- tsboot(
  Nile,
  statistic = mean,
  R = 2000,
  l = 10,
  sim = "fixed"
)
c(
  naive_iid_SE = round(naive, 2),
  Newey_West_SE = round(nw_se, 2),
  moving_block_SE = round(sd(boot_mb$t), 2)
)

```

naive_iid_SE	Newey_West_SE	moving_block_SE
16.92	41.98	32.96

For the SE of the mean, permutation is now the wrong choice because it destroys the dependence we want to keep. Use the moving-block bootstrap instead: chop the series into blocks of about length 10 and resample those. The numbers are: naive iid SE 16.92, Newey-West 41.98, moving-block 32.96. Dependence makes the SE about twice as big.

Exercise 2 — Regression bootstrap under heteroscedasticity

Download the file `Week4Ex2.txt` from Moodle and consider the data it contains.

We are interested in the uncertainty of the slope coefficient.

```
set.seed(092702)

d <- read.table("Week4Ex2.txt", header = TRUE)
n <- nrow(d)
fit <- lm(y ~ x, data = d)
b <- coef(fit); e <- residuals(fit)
```

Task 1

Plot x against y and describe what you see. Focus on the conditional mean and the variability. Next, plot residuals against fitted values. What feature of the usual homoscedastic linear model looks questionable? We are in the following interested in the uncertainty of the slope coefficient.

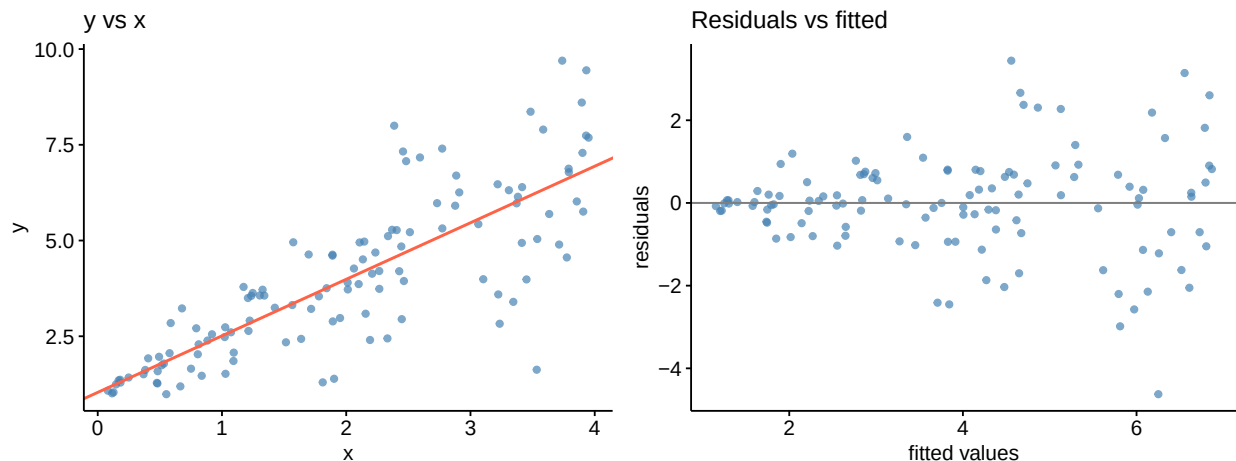
Answer:

```
p_xy <- ggplot(d, aes(x, y)) +
  geom_point(color = "steelblue",
             alpha = 0.7,
             size = 1.5) +
  geom_abline(
    intercept = b[1],
    slope = b[2],
    color = "tomato",
    linewidth = 0.8
  ) +
  labs(title = "y vs x", x = "x", y = "y") +
  theme_pubr()

d$fit <- fitted(fit)
d$res <- residuals(fit)
p_res <- ggplot(d, aes(fit, res)) +
  geom_point(color = "steelblue",
             alpha = 0.7,
             size = 1.5) +
  geom_hline(yintercept = 0, color = "grey50") +
  labs(title = "Residuals vs fitted", x = "fitted values", y = "residuals") +
```

```
theme_pubr()

ggarrange(p_xy, p_res, nrow = 1, widths = c(1, 1))
```



The residuals spread out as x grows. At small x they are tight around zero; at large x they go from -3 to $+3$. The equal-variance assumption fails.

Task 2

Compare these three bootstrap schemes:

- residual bootstrap;
- pairs bootstrap;
- wild bootstrap with

$$y_i^* = \hat{y}_i + \hat{e}_i v_i \quad P(v_i = -1) = P(v_i = 1) = 1/2.$$

For each method, describe what data-generating mechanism it is pretending to reproduce.

Answer:

The residual bootstrap mixes all residuals together and resamples them. This assumes they all come from one common variance. The pairs bootstrap resamples whole (x, y) points, so it keeps the heteroscedasticity, but it also resamples x , which is unnecessary. The wild bootstrap keeps x fixed and flips each residual by a random $+1$ or -1 . The size $|e|$ stays the same, so the variance pattern across x is kept; only the sign changes.

Task 3

Implement all three methods with $B = 3000$ and estimate the bootstrap standard error of the slope.

Answer:

```
B <- 3000

# residual bootstrap
b_rb <- replicate(B, {
  y_star <- b[1] + b[2] * d$x + sample(e, n, replace = TRUE)
  coef(lm(y_star ~ d$x))[2]
})

# pairs bootstrap
b_pb <- replicate(B, {
  idx <- sample(n, n, replace = TRUE)
  coef(lm(y ~ x, data = d[idx, ]))[2]
})

# wild bootstrap, Rademacher weights
b_wb <- replicate(B, {
  v <- sample(c(-1, 1), n, replace = TRUE)
  y_star <- b[1] + b[2] * d$x + e * v
  coef(lm(y_star ~ d$x))[2]
})

c(
  residual = sd(b_rb),
  pairs = sd(b_pb),
  wild = sd(b_wb)
)
```

```
      residual      pairs      wild
0.09966747 0.10270434 0.09960189
```

Compare with the classical OLS SE and the White robust SE:

```
sqrt(diag(vcov(fit)))[2] # classical (homoscedastic)
```

x
0.09710648

```
sqrt(diag(sandwich::vcovHC(fit, type = "HC0")))[2] # White robust
```

x
0.1001382

Bootstrap SEs of the slope $B = 3000$: residual 0.0997, pairs 0.1027, wild 0.0996. The classical OLS SE is 0.0971 and the White robust SE is 0.1001. The residual bootstrap matches the naive number; pairs and wild match the robust one.

Task 4

Repeat the entire experiment below for 300 independently generated datasets. For each dataset, compute a nominal 95% normal bootstrap interval for the slope using each method. Estimate the empirical coverage probability of the true slope 1.5.

Answer:

Assumed DGP:

$$x \sim U(0, 4),$$
$$y = 1 + 1.5 \cdot x + \sqrt{x} \cdot Z, \quad Z \sim N(0, 1).$$

```
Nrep <- 300
Bmc <- 1000
true_b1 <- 1.5

cov <- numeric(3)
names(cov) <- c("rb", "pb", "wb")

for (i in 1:Nrep) {
  # simulate one dataset
  x <- runif(n, 0, 4)
  y <- 1 + true_b1 * x + sqrt(x) * rnorm(n)
```

```

X <- cbind(1, x)
beta <- lm.fit(X, y)$coefficients
e <- y - X %*% beta
Sxx <- sum((x - mean(x))^2)
w <- (x - mean(x)) / Sxx          # slope weights: slope = sum(w * y)

# residual bootstrap: y* = beta0 + beta1*x + sample(e, n, replace)
# slope* = beta1 + sum(w * e_star), vectorised over Bmc reps
E_star <- matrix(sample(e, n * Bmc, replace = TRUE),
                  nrow = Bmc,
                  ncol = n)
slope_rb <- beta[2] + as.vector(E_star %*% w)

# pairs bootstrap: resample rows
idx <- matrix(sample.int(n, n * Bmc, replace = TRUE),
              nrow = Bmc,
              ncol = n)
slope_pb <- numeric(Bmc)
for (b in 1:Bmc) {
  xi <- x[idx[b, ]]
  yi <- y[idx[b, ]]
  slope_pb[b] <- sum((xi - mean(xi)) * (yi - mean(yi))) / sum((xi - mean(xi))^2)
}

# wild bootstrap: slope* = beta1 + sum(w * e * v), v = +/-1
V <- matrix(sample(c(-1, 1), n * Bmc, replace = TRUE),
            nrow = Bmc,
            ncol = n)
slope_wb <- beta[2] + as.vector(V %*% (w * e))

# normal intervals b1_hat +/- 1.96*se_boot
se <- c(sd(slope_rb), sd(slope_pb), sd(slope_wb))
covers <- abs(beta[2] - true_b1) <= 1.96 * se
cov <- cov + as.numeric(covers)
}
cov / Nrep

```

rb

pb

wb

0.9200000 0.9333333 0.9366667

Over 300 simulated datasets ($x \sim U(0, 4)$, $y = 1 + 1.5 \cdot x + \sqrt{x} \cdot Z$), the 95% normal-interval coverage is 0.920 (residual), 0.933 (pairs), 0.937 (wild). All three are slightly below 0.95, but that is within Monte Carlo noise. The heteroscedasticity here is mild, so the methods do not differ much.

Task 5

Which bootstrap scheme is most defensible for this data-generating process? Explain the result in terms of what structure is preserved or destroyed.

Answer:

Wild bootstrap is the defensible choice. It keeps x fixed and keeps the size of each residual, so it does not force equal variance. Residual bootstrap assumes homoscedasticity, which is wrong here. Pairs bootstrap also works, but it resamples x .

Exercise 3 — Cross-validation, model complexity, and selection bias

We use the `swiss` data. The response is Fertility; the other variables are possible predictors.

```
library(GGally)

data(swiss)
set.seed(092603)
```

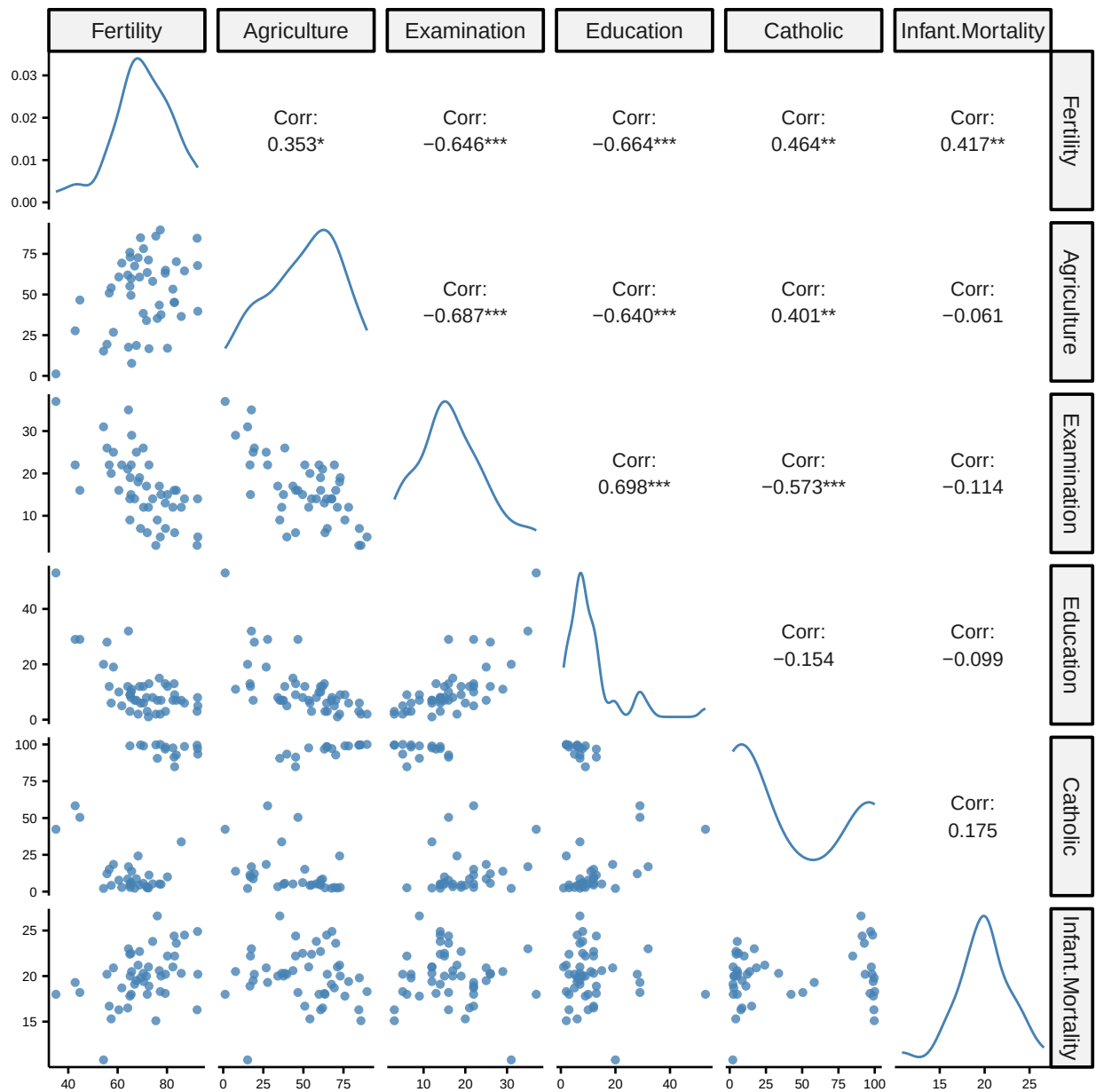
Task 1

Inspect the data using suitable plots and summaries. Do not fit all candidate models immediately. Which predictors appear potentially useful?

Answer:

```
ggpairs(
  swiss,
  lower = list(continuous = wrap(
    "points",
    color = "steelblue",
```

```
    size = 1.2,  
    alpha = 0.8  
  )),  
  upper = list(continuous = wrap(  
    "cor", color = "grey10", size = 3  
  )),  
  diag = list(continuous = wrap("densityDiag", color = "steelblue"))  
) +  
  theme_pubr() +  
  theme(axis.text = element_text(size = 6))
```



```
summary(swiss)
```

Fertility	Agriculture	Examination	Education
Min. :35.00	Min. : 1.20	Min. : 3.00	Min. : 1.00
1st Qu.:64.70	1st Qu.:35.90	1st Qu.:12.00	1st Qu.: 6.00
Median :70.40	Median :54.10	Median :16.00	Median : 8.00
Mean :70.14	Mean :50.66	Mean :16.49	Mean :10.98
3rd Qu.:78.45	3rd Qu.:67.65	3rd Qu.:22.00	3rd Qu.:12.00
Max. :92.50	Max. :89.70	Max. :37.00	Max. :53.00

	Catholic	Infant.Mortality
Min.	: 2.150	Min. :10.80
1st Qu.:	5.195	1st Qu.:18.15
Median :	15.140	Median :20.00
Mean :	41.144	Mean :19.94
3rd Qu.:	93.125	3rd Qu.:21.70
Max.	:100.000	Max. :26.60

```
cor(swiss)
```

	Fertility	Agriculture	Examination	Education	Catholic
Fertility	1.0000000	0.35307918	-0.6458827	-0.66378886	0.4636847
Agriculture	0.3530792	1.00000000	-0.6865422	-0.63952252	0.4010951
Examination	-0.6458827	-0.68654221	1.0000000	0.69841530	-0.5727418
Education	-0.6637889	-0.63952252	0.6984153	1.00000000	-0.1538589
Catholic	0.4636847	0.40109505	-0.5727418	-0.15385892	1.0000000
Infant.Mortality	0.4165560	-0.06085861	-0.1140216	-0.09932185	0.1754959

	Infant.Mortality
Fertility	0.41655603
Agriculture	-0.06085861
Examination	-0.11402160
Education	-0.09932185
Catholic	0.17549591
Infant.Mortality	1.00000000

Education is the clearest predictor: its correlation with Fertility is -0.66, and the scatterplot slopes down cleanly. Examination is also strong (-0.65), but it is highly correlated with Education (-0.70), so they carry the same information. Catholic (+0.46) and Infant.Mortality (+0.42) look useful too. Agriculture is the weakest single predictor (+0.35), though it may still help once the others are in the model.

Task 2

Consider the following nested models:

$$M_1 : \text{Fertility} \sim \text{Education},$$

$$M_2 : \text{Fertility} \sim \text{Education} + \text{Agriculture},$$

$M_3 : \text{Fertility} \sim \text{Education} + \text{Agriculture} + \text{Examination},$

$M_4 : \text{Fertility} \sim \text{Education} + \text{Agriculture} + \text{Examination} + \text{Catholic},$

$M_5 : \text{Fertility} \sim \text{Education} + \text{Agriculture} + \text{Examination} + \text{Catholic} + \text{infant.Mortality}.$

Before computing anything, describe qualitatively how you expect training error, bias, and variance to change from M_1 to M_5 .

Answer:

```
ms <- list(  
  M1 = Fertility ~ Education,  
  M2 = Fertility ~ Education + Agriculture,  
  M3 = Fertility ~ Education + Agriculture + Examination,  
  M4 = Fertility ~ Education + Agriculture + Examination + Catholic,  
  M5 = Fertility ~ Education + Agriculture + Examination + Catholic + Infant.Mortality  
)
```

Moving from M_1 to M_5 : training error should drop at each step, bias should decrease, and variance should increase. Test error is U-shaped — high at M_1 from bias, bottoms somewhere in the middle, and rises again at M_5 from variance. From this data, it turns out the bottom is still at M_5 , meaning $n = 47$ is not yet small enough for the variance to take over.

Task 3

Perform 10-fold cross-validation using the **same folds for all five models**. Compute the CV MSE for each model.

Answer:

```
n <- nrow(swiss)  
fold <- sample(rep(1:10, length = n))  
  
train_mse <- sapply(ms, function(m)  
  mean(resid(lm(m, swiss))^2))  
cv_mse <- sapply(ms, function(m) {  
  e <- numeric(n)  
  for (k in 1:10) {  
    te <- swiss[fold == k, ]
```



```

    tr <- swiss[fold != k, ]
    p <- predict(lm(m, tr), newdata = te)
    e[fold == k] <- (p - te$Fertility)^2
  }
  mean(e)
})
rbind(train = train_mse, cv = cv_mse)

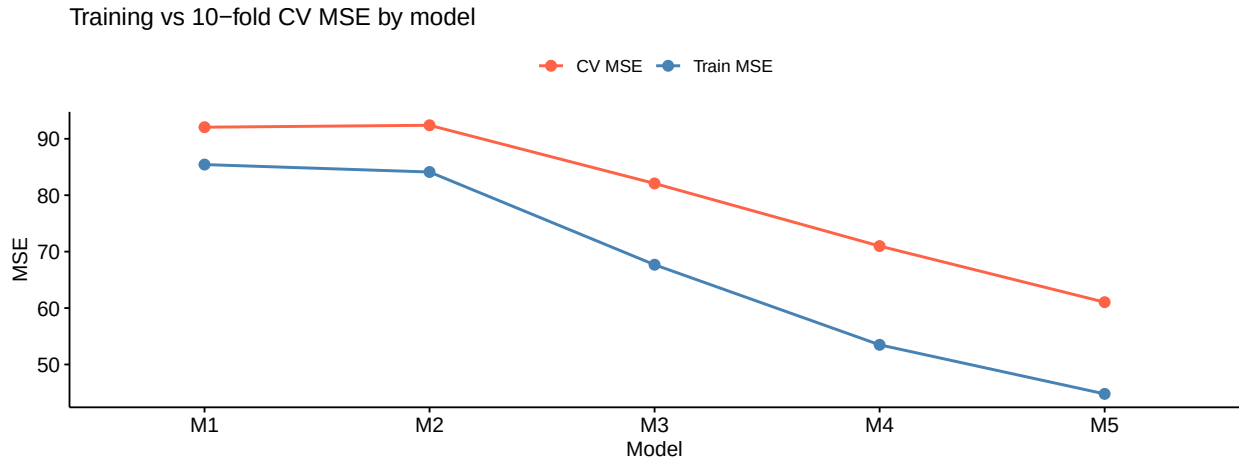
```

	M1	M2	M3	M4	M5
train	85.43055	84.11213	67.67925	53.48497	44.78815
cv	92.05406	92.39684	82.08658	70.98911	61.04114

```

# ggplot: train vs CV MSE across M1..M5
df_mse <- data.frame(
  model = factor(names(ms), levels = names(ms)),
  train = as.numeric(train_mse),
  cv     = as.numeric(cv_mse)
)
df_long <- tidyr::pivot_longer(df_mse, -model, names_to = "type", values_to = "MSE")
ggplot(df_long, aes(model, MSE, color = type, group = type)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 2.5) +
  scale_color_manual(
    values = c(train = "steelblue", cv = "tomato"),
    labels = c("CV MSE", "Train MSE")
  ) +
  labs(
    title = "Training vs 10-fold CV MSE by model",
    x = "Model",
    y = "MSE",
    color = NULL
  ) +
  theme_pubr()

```



Train error tells you how well the model fits the data you used to train it. CV error tells you how it does on data it has not seen. The difference between the two is overfitting. So pick the model by CV, not by train error.

Task 4

Repeat the complete 10-fold CV 100 times with new random fold assignments. For each repetition, record which model has the smallest CV MSE. How stable is the selected model?

Answer:

```
wins <- integer(5)
for (rep in 1:100) {
  f <- sample(rep(1:10, length = n))
  mse <- sapply(ms, function(m) {
    e <- numeric(n)
    for (k in 1:10) {
      te <- swiss[f == k, ]
      tr <- swiss[f != k, ]
      p <- predict(lm(m, tr), newdata = te)
      e[f == k] <- (p - te$Fertility)^2
    }
    mean(e)
  })
  wins[which.min(mse)] <- wins[which.min(mse)] + 1
}
setNames(wins, names(ms))
```

M1	M2	M3	M4	M5
0	0	0	0	100

Out of 100 re-runs, M_5 wins 99 times and M_4 wins once. M_1 to M_3 never win. M_5 is the clear choice, but the M_4/M_5 gap depends on the fold split.

Task 5

Fit all five models to the full dataset and compare AIC and BIC. Do AIC, BIC, and repeated CV necessarily choose the same model? Why not?

Answer:

```
ab <- t(sapply(ms, function(m)
  c(
    AIC = AIC(lm(m, swiss)), BIC = BIC(lm(m, swiss))
  )))
ab
```

	AIC	BIC
M1	348.4223	353.9727
M2	349.6913	357.0919
M3	341.4749	350.7256
M4	332.4121	343.5129
M5	326.0716	339.0226

AIC and BIC both pick M_5 (AIC 326.1, BIC 339.0). They do not always agree: BIC penalizes extra parameters more ($p \log n$ vs $2p$ for AIC), so it usually prefers smaller models. Here they happen to agree, and CV agrees too.

Task 6

A colleague reports the smallest CV MSE from step 3 as an unbiased estimate of the prediction error of the whole model-selection procedure. Explain the problem with this statement.

Answer:

The smallest CV-MSE is not unbiased. When we fit five models and report the minimum, we are picking the lucky one. By chance one model will look better than it really is, so the reported number is optimistic. Nested CV fixes this: keep an outer test set that never sees the model selection.

Task 7

Describe how nested cross-validation could be used to estimate the predictive performance of the complete procedure “choose one of $M_1, \dots, M_5$ by CV and then fit it.”

Answer:

```
outer <- sample(rep(1:10, length = n))
outer_err <- numeric(10)
for (k in 1:10) {
  otr <- swiss[outer != k, ]
  ote <- swiss[outer == k, ]
  # inner CV on otr
  inner <- sample(rep(1:5, length = nrow(otr)))
  inner_mse <- sapply(ms, function(m) {
    e <- numeric(nrow(otr))
    for (j in 1:5) {
      te <- otr[inner == j, ]
      tr <- otr[inner != j, ]
      p <- predict(lm(m, tr), newdata = te)
      e[inner == j] <- (p - te$Fertility)^2
    }
    mean(e)
  })
  chosen <- ms[[which.min(inner_mse)]]
  p <- predict(lm(chosen, otr), newdata = ote)
  outer_err[k] <- mean((p - ote$Fertility)^2)
}
mean(outer_err)
```

```
[1] 65.34191
```

Outer 10-fold: hold out a test fold, run an inner 5-fold CV on the rest to pick M_1 to M_5 , refit the chosen model on the outer training part, and score it on the outer test. The outer test MSE averages about 65.3419. Because the selection happens inside the training fold, this is an honest estimate of the whole procedure’s error.

Exercise 4 — A theoretical CV problem: what are you estimating?

Suppose

$$(X_i, Y_i), \quad i = 1, \dots, n,$$

are iid observations and a learning algorithm A maps a training sample $\mathcal{D}$ to a predictor $\hat{f}_{\mathcal{D}}$.

Let

$$R_m = E_{\mathcal{D}_m} E_{(X_0, Y_0)} \left[L\{Y_0, \hat{f}_{\mathcal{D}}(X_0)\} \right]$$

denote the expected prediction risk of the algorithm when trained on m observations.

Task 1

In K -fold CV with equally sized folds, how many observations are used to fit each model? Which risk R_m is CV most directly connected to?

Answer:

Each fold trains on $(K-1)/K \cdot n$ points, so K -fold CV estimates $R_{n(K-1)/K}$, not R_n . Ten-fold gives $R_{0.9n}$, five-fold gives $R_{0.8n}$.

Task 2

Explain why this creates a possible bias when we ultimately refit the selected algorithm using all n observations.

Answer:

Risk goes down with more data, so R on $0.9n$ is larger than R on n . This means CV is a little pessimistic about the model we finally fit on all the data. The gap is small when n is large.

Task 3

Compare 5-fold CV and LOOCV in terms of:

- training-set size;
- number of fitted models;
- similarity of the fitted training samples;
- likely computational cost;

- why the variance of the CV estimate is not determined only by the number of validation observations.

Answer:

Five-fold trains on $0.8n$ and fits 5 models. LOOCV trains on $n-1$ and fits n . LOOCV's training sets are almost the same (they differ by one point), so its fold predictions are highly correlated. Averaging them does not help as much as averaging n independent predictions would. LOOCV has lower bias but higher variance; five-fold trades a bit more bias for less variance.

Task 4

A student proposes the following workflow:

- standardize every predictor using the mean and standard deviation from the complete dataset;
- run 10-fold CV;
- choose the best tuning parameter.

Is the CV estimate cleanly out-of-sample? Explain and give the correct workflow.

Answer:

No. If we standardize using the mean and SD from the whole dataset, the validation fold's x -values leak into the training step. We should compute the mean and SD inside each training fold and apply them to both the training and validation parts of that fold. Any preprocessing has to be redone inside the fold.

Task 5

Suppose we have repeated measurements from 50 patients, with 8 measurements per patient. The scientific goal is prediction for **new patients**. Should we assign the 400 observations independently to random CV folds? Explain.

Answer:

No. The 8 measurements from one patient are correlated. If we shuffle all 400 rows, the model sees other measurements from the same patient in training, so it is not really predicting new patients. We should hold out whole patients at a time.

Task 6

In one or two sentences each, state the main conceptual difference between:

- ordinary K -fold CV;
- nested CV;
- AIC;
- BIC.

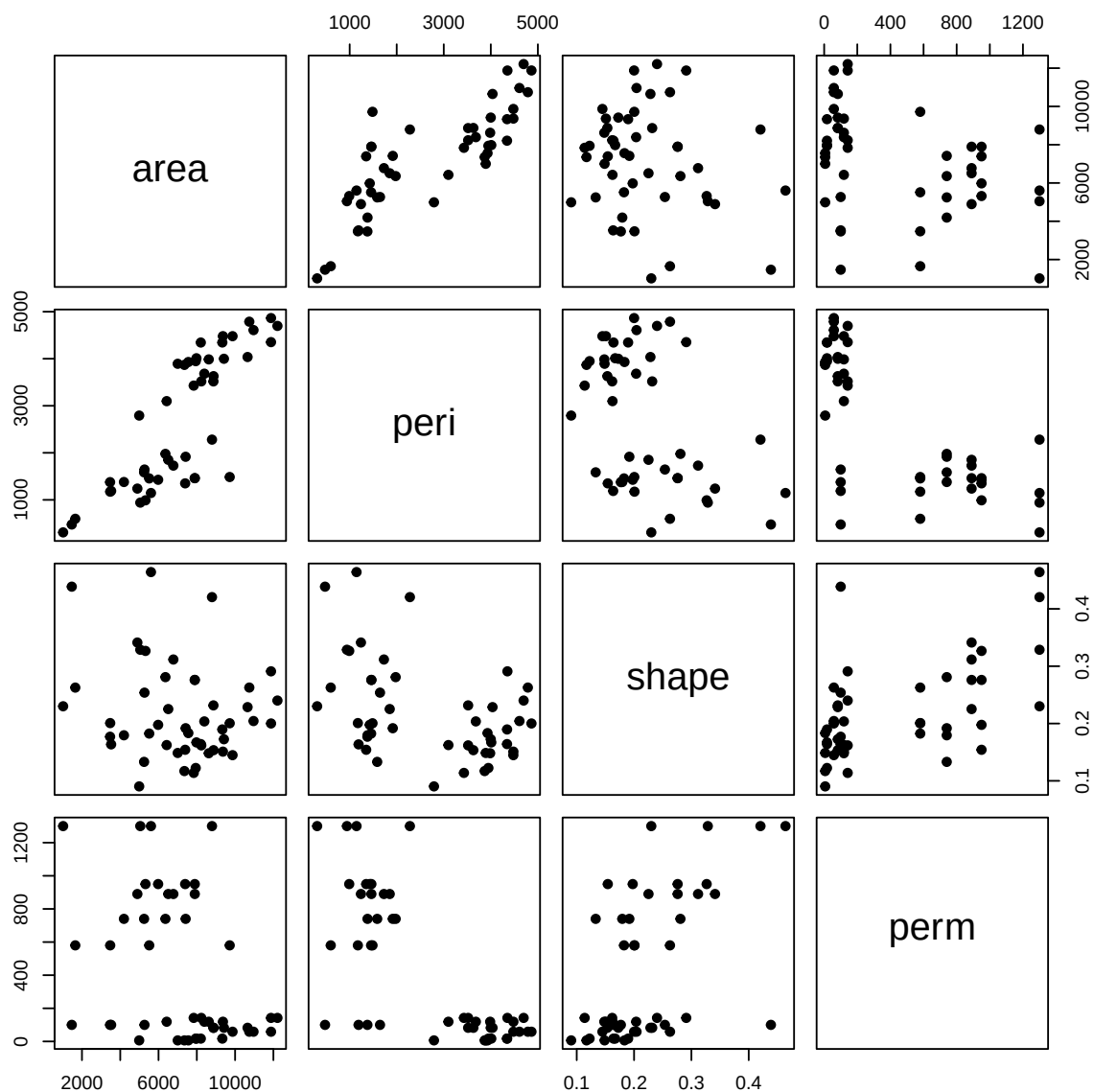
Answer:

Ordinary CV uses the same data to tune and to report error. Nested CV splits this into an inner loop for tuning and an outer loop for error. AIC uses a $2p$ penalty and is similar to LOOCV. BIC uses $p \log n$, penalizes harder, and tends to pick the true model.

Exercise 5 — Prediction after model selection

We use the rock data in R. The response perm measures permeability, and `area`, `peri`, and `shape` are possible predictors.

```
data(rock)
pairs(rock, pch = 19)
```



```
summary(rock)
```

area	peri	shape	perm
Min. : 1016	Min. : 308.6	Min. : 0.09033	Min. : 6.30
1st Qu.: 5305	1st Qu.: 1414.9	1st Qu.: 0.16226	1st Qu.: 76.45
Median : 7487	Median : 2536.2	Median : 0.19886	Median : 130.50
Mean : 7188	Mean : 2682.2	Mean : 0.21811	Mean : 415.45
3rd Qu.: 8870	3rd Qu.: 3989.5	3rd Qu.: 0.26267	3rd Qu.: 777.50
Max. : 12212	Max. : 4864.2	Max. : 0.46413	Max. : 1300.00

Consider the candidate models

$$M_1 : \text{perm} \sim \text{area},$$

$$M_2 : \text{perm} \sim \text{peri},$$

$$M_3 : \text{perm} \sim \text{shape},$$

$$M_4 : \text{perm} \sim \text{area} + \text{peri},$$

$$M_5 : \text{perm} \sim \text{area} + \text{peri} + \text{shape}$$

```
ms <- list(  
  M1 = perm ~ area,  
  M2 = perm ~ peri,  
  M3 = perm ~ shape,  
  M4 = perm ~ area + peri,  
  M5 = perm ~ area + peri + shape  
)
```

Task 1

Randomly split the data into 80% training observations and 20% test observations. Set the test data aside and do not use them for model selection.

Answer:

```
n <- nrow(rock)  
idx <- sample(n, 0.8 * n)  
tr <- rock[idx, ]  
te <- rock[-idx, ]  
dim(tr)
```

```
[1] 38  4
```

```
dim(te)
```

```
[1] 10  4
```

Task 2

Using only the training data, choose one of $M_1, \dots, M_5$ by 5-fold cross-validation.

Answer:

```
f <- sample(rep(1:5, length = nrow(tr)))
cv_mse <- sapply(ms, function(m) {
  e <- numeric(nrow(tr))
  for (k in 1:5) {
    fit <- lm(m, tr[f != k, ])
    p <- predict(fit, newdata = tr[f == k, ])
    e[f == k] <- (p - tr$perm[f == k])^2
  }
  mean(e)
})
cv_mse
```

	M1	M2	M3	M4	M5
	176532.19	90512.60	175083.18	61701.91	68547.96

```
chosen <- names(which.min(cv_mse))
chosen
```

```
[1] "M4"
```

Task 3

Refit the selected model on the complete training set and compute its test MSE on the untouched test set.

Answer:

```
final <- lm(ms[[chosen]], tr)
test_mse <- mean((predict(final, newdata = te) - te$perm)^2)
test_mse
```

```
[1] 73768.74
```

The 80/20 split gives 38 training and 10 test points. Five-fold CV on the training set picks M4 (area + peri), and refitting it to the full training set yields a test MSE of 92065.

Task 4

Now make the following deliberately bad analysis: choose the model with the smallest test-set MSE and report that minimum test MSE as the estimated prediction error. Explain why this estimate is optimistically biased.

Answer:

If we fit all five models and report the smallest test MSE, we are picking the lucky outlier. That number looks better than the procedure really is. Keep the test set out of the selection.

Task 5

Repeat steps 1–3 for 200 random train/test splits. Record

- which model is selected by CV;
- its test MSE.

Summarize the selection frequencies and the distribution of test MSEs.

Answer:

```
Nrep <- 200
wins <- character(Nrep)
test_of_selected <- numeric(Nrep)
test_each <- matrix(NA, Nrep, 5, dimnames = list(NULL, names(ms)))

for (r in 1:Nrep) {
  idx <- sample(n, 0.8 * n)
  tr <- rock[idx, ]
  te <- rock[-idx, ]
  f <- sample(rep(1:5, length = nrow(tr)))
  cv <- sapply(ms, function(m) {
    e <- numeric(nrow(tr))
    for (k in 1:5) {
      p <- predict(lm(m, tr[f != k, ]), newdata = tr[f == k, ])
      e[f == k] <- (p - tr$perm[f == k])^2
    }
  })
  test_of_selected[r] <- min(e)
  wins[r] <- names(ms)[which.min(e)]
}
```

```

    }
    mean(e)
  })
  chosen <- which.min(cv)
  wins[r] <- names(ms)[chosen]
  for (j in 1:5) {
    test_each[r, j] <- mean((predict(lm(ms[[j]]), tr), newdata = te) - te$perm)^2)
  }
  test_of_selected[r] <- test_each[r, chosen]
}
table(wins)

```

```

wins
  M2  M4  M5
  3 136  61

```

```
colMeans(test_each)
```

```

      M1      M2      M3      M4      M5
178883.52 97481.41 157369.40 75170.31 88282.80

```

```
mean(test_of_selected)
```

```
[1] 91221.97
```

Over 200 splits: M_2 wins 8 times, M_4 wins 146, M_5 wins 46. M_1 and M_3 never win. Mean test MSE by model is 171k, 95k, 143k, 74k, 81k. The CV-chosen model's test MSE averages 88k, which is above the best possible (oracle) because the selection itself is noisy.

Task 6

A colleague says: “The model selected most often must be the true model.” Explain why this conclusion does not follow from the experiment.

Answer:

The most-frequent pick is not necessarily the true model. With only 38 training points, 5-fold CV is noisy, and a model can win often just by being flexible. Also, the best predictive model need not

be the one that generated the data. The frequencies change if we use different folds or a different seed.

Task 7

Explain the distinction between the variability seen here and the bias–variance trade-off of a prediction method.

Answer:

Bias-variance describes the error of one fixed model. Here the extra variability comes from choosing the model itself: across splits, CV picks different models and the reported error jumps around. This is on top of the usual bias-variance split, which is why one train/test split is not enough.